Chapter 10. Attention and Transformers

Lecture 11/2: attention-based captioning, spatial and channel attention, inductive bias, scaled dot-product, cross- and self-attention, positional encoding, masking, multi-head attention, Transformer encoder, layer normalization

Contents

1 Attention-Based Image Captioning **2**

2 Spatial and Channel Attention **3**

3 Inductive Bias **3**

4 The General Attention Layer **4**

4.1 One query 4

4.2 Why divide by $\sqrt{D}$ 4

4.3 Many queries, keys and values 5

5 Permutation Symmetry **6**

6 Positional Encoding **6**

7 Masked Self-Attention **7**

8 Multi-Head Attention **8**

9 The Transformer Encoder **8**

10 Layer Normalization **9**

11 Why Transformers **9**

1 Attention-Based Image Captioning

Motivation. Naive captioning (Chapter 9) feeds one global image vector once, and a strong language model learns to ignore it. Attention lets each generated word look at the image regions it describes, by recomputing a weighted summary of the spatial feature map at every step.

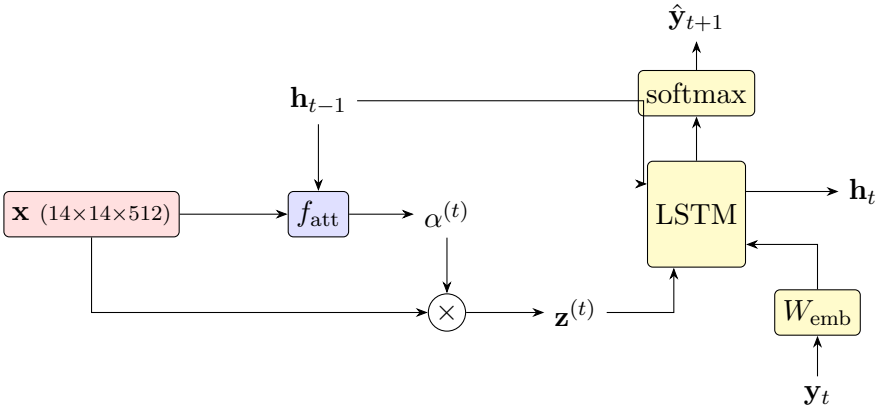
Instead of the fully connected feature vector, keep the last convolutional feature map of the pretrained CNN: for a $224 \times 224 \times 3$ input, conv5 of VGG has shape $14 \times 14 \times 512$. Write it as $L = 14 \cdot 14 = 196$ feature vectors $\mathbf{x}_{i,j} \in \mathbb{R}^{512}$, one per spatial location. An **attention map** for a word is a nonnegative matrix $\alpha^{(t)} \in \mathbb{R}^{14\times 14}$ with the same spatial size as the feature map and $\sum_{i,j} \alpha^{(t)}_{i,j} = 1$. There is one per generated word ($\alpha^{(1)}$ for "a", $\alpha^{(2)}$ for "bird", ...), and it summarizes the image as $\mathbf{z}^{(t)} = \sum_{i,j} \alpha^{(t)}_{i,j} \mathbf{x}_{i,j} \in \mathbb{R}^{512}$. In sizes: $\mathbf{x} \in \mathbb{R}^{14\times 14\times 512}$, $\mathbf{x}_{i,j} \in \mathbb{R}^{512}$, $\alpha^{(t)} \in \mathbb{R}^{14\times 14}$. Two questions follow: how to compute the attention, and how to use it to predict the word.

Definition 10.1: attention-based captioning (Show, Attend and Tell)

At step t, with previous hidden state $\mathbf{h}_{t-1}$ and current input word $\mathbf{y}_t$:

$$
\begin{aligned}
e^{(t)} &= f_{\text{att}}(\mathbf{x}, \mathbf{h}_{t-1}) && \text{(alignment scores, one per location)},\\
\alpha^{(t)}_{i,j} &= \frac{\exp(e^{(t)}_{i,j})}{\sum_{i',j'} \exp(e^{(t)}_{i',j'})} && \text{(attention weights)},\\
\mathbf{z}^{(t)} &= \sum_{i,j} \alpha^{(t)}_{i,j} \mathbf{x}_{i,j} && \text{(context vector)},\\
\mathbf{h}_t &= \text{LSTM}(\mathbf{h}_{t-1}, W_{\text{emb}}\mathbf{y}_t, \mathbf{z}^{(t)}) && \text{(modified LSTM)},\\
\hat{\mathbf{y}}_{t+1} &\propto \exp\left(W^o(W_{\text{emb}}\mathbf{y}_t + W^h\mathbf{h}_t + W^z\mathbf{z}^{(t)})\right) && \text{(next-word distribution)}.
\end{aligned}
$$

The modified LSTM takes the context vector as an extra input to every gate. The attention module f_{att} is a small MLP. The cell is repeated N times, once per generated word.



One step of attention-based captioning: the attention module turns $(\mathbf{x}, \mathbf{h}_{t-1})$ into weights $\alpha^{(t)}$, and the weighted feature $\mathbf{z}^{(t)}$ enters the LSTM.

The weights $\alpha^{(t)}_{i,j}$ are positive and sum to 1, so $\mathbf{z}^{(t)}$ is a weighted average of the location features. "No attention" corresponds to uniform attention, every weight equal to $1/L$, which reduces to the naive model with the average-pooled feature.

Soft versus hard attention. The rule above is **soft attention**: a deterministic weighted average, differentiable in all parameters and trained by backpropagation. **Hard attention** instead samples one location $s_t \sim \text{Categorical}(\alpha^{(t)})$ and sets $\mathbf{z}^{(t)} = \mathbf{x}_{s_t}$. In the lecture's visualizations, soft attention

spreads over the bird and the water, while hard attention picks one spot per word ("bird", "water"). The sampling step of hard attention has no gradient with respect to α, so hard attention is trained with score-function (REINFORCE-type) gradient estimators rather than plain backpropagation.

2 Spatial and Channel Attention

Attention weights can be placed on different axes of a feature map $U \in \mathbb{R}^{H \times W \times C}$.

- **Spatial attention** weights the $H \times W$ locations (Section 1).
- **Channel attention** weights the C channels.

Definition 10.2: squeeze-and-excitation block (SENet)

Given $U = [\mathbf{u}_1, \ldots, \mathbf{u}_C]$ with $\mathbf{u}_c \in \mathbb{R}^{H \times W}$:

$$
\begin{aligned}
s_c &= \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i,j) && \text{(squeeze: global average pooling)}, \\
\mathbf{e} &= \sigma\big(W_2 \, \mathrm{ReLU}(W_1 \mathbf{s})\big) && \text{(excitation)}, \\
\tilde{\mathbf{u}}_c &= e_c \, \mathbf{u}_c && \text{(scale)},
\end{aligned}
$$

with $W_1 \in \mathbb{R}^{(C/r) \times C}$ and $W_2 \in \mathbb{R}^{C \times (C/r)}$. In words: global pooling, then a channel-wise weight, then multiplication with the original map.

Each $e_c \in (0, 1)$ comes from a sigmoid, not a softmax, so here the attention weights need not sum to 1. The block adds $2C^2/r$ weights (reduction ratio r, e.g. 16) and is inserted into residual blocks (SE-ResNet) or mobile blocks (ANTNets).

3 Inductive Bias

Motivation. We now have MLPs, CNNs, RNNs and attention. Why different models for different data? Each architecture builds in assumptions about its data, and attention is the one with the fewest.

So far we have seen four families:

- MLP: linear layers and nonlinear functions.
- CNN: convolution, pooling, nonlinearities and an MLP head.
- RNN: MLPs for the recurrent update, plus gating functions.
- Attention and Transformers: dot-product attention plus MLPs.

One can also mix data and models: a CNN for sequences (one-dimensional convolution), an RNN for images (scanning pixels in order), an MLP for images (flattened pixels).

Definition 10.3: inductive bias

The **inductive bias** of a learner is the set of assumptions it uses to predict outputs for inputs it has not observed.

Most architectural biases concern **locality** and **invariance**.

- Convolution exploits the 2D grid structure and translation equivariance (Chapter 7).
- An RNN exploits a 1D chain and a first-order Markov assumption: $\mathbf{h}_t$ depends on the past only through $\mathbf{h}_{t-1}$.

These assumptions make models efficient, but they often do not transfer across data modalities. The goal is one model for images, point clouds, audio and language. The key requirement is to make few assumptions about the input structure.

4 The General Attention Layer

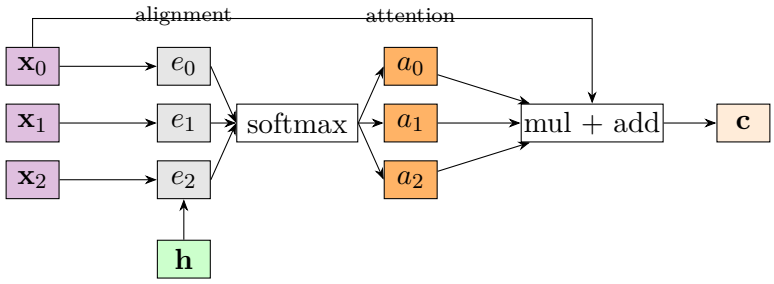
4.1 One query

Definition 10.4: attention with one query

Inputs: vectors $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$ (stacked as $\mathbf{x} \in \mathbb{R}^{N \times D}$) and a query $\mathbf{h} \in \mathbb{R}^D$.

$$\begin{aligned} \text{alignment:} \quad & e_i = f_{\text{att}}(\mathbf{h}, \mathbf{x}_i) = \frac{\mathbf{h} \cdot \mathbf{x}_i}{\sqrt{D}}, \\ \text{attention:} \quad & \mathbf{a} = \text{softmax}(\mathbf{e}), \\ \text{output:} \quad & \mathbf{c} = \sum_{i=1}^{N} a_i \mathbf{x}_i \in \mathbb{R}^D \quad \text{(context vector)}. \end{aligned}$$

A feature map of size $H \times W \times D$ is stretched into $N = HW$ vectors. The operation does not care about the order of the features: reordering $\mathbf{x}_1, \dots, \mathbf{x}_N$ leaves $\mathbf{c}$ unchanged, since $\mathbf{c}$ is a sum. The slides call this **permutation invariance**.



The general attention layer with one query $\mathbf{h}$*: scores, softmax, then a weighted sum of the inputs.*

4.2 Why divide by $\sqrt{D}$

The lecture replaced a general f_{att} by a *scaled* dot product. The reasoning: in higher dimension the dot product has more terms, so the logits have larger variance. Large logits give a peaked, low-entropy softmax that puts almost all weight on one input. Dividing by $\sqrt{D}$ removes the growth.

In the lecture's computation, if the entries of $\mathbf{h}$ and $\mathbf{x}$ are independent with mean 0 and variance 1, then $z = \mathbf{h} \cdot \mathbf{x} = \sum_{m=1}^{D} h_m x_m$ has $\mathbb{E}z = 0$ and $\operatorname{Var} z = \sum_m \operatorname{Var}(h_m x_m) = D$. So the logits have standard deviation $\sqrt{D}$, and dividing by $\sqrt{D}$ restores unit variance for every D. For large D without scaling, the largest logit is typically far above the others, the softmax is nearly one-hot, and almost no gradient passes through the attention weights.

4.3 Many queries, keys and values

Definition 10.5: multiple queries

With queries $\mathbf{q}_1, \dots, \mathbf{q}_M \in \mathbb{R}^D$ (stacked as $\mathbf{q} \in \mathbb{R}^{M \times D}$), each query produces its own context vector:

$$e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{x}_i}{\sqrt{D}}, \qquad \mathbf{a}_{\cdot,j} = \operatorname{softmax}(\mathbf{e}_{\cdot,j}), \qquad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{x}_i.$$

Here the inputs $\mathbf{x}$ play two roles, in the alignment and in the output. Adding a separate fully connected layer before each role makes the layer more expressive, and lets the input and output dimensions change.

Definition 10.6: cross-attention layer

With learnable $W_k \in \mathbb{R}^{D \times D_k}$ and $W_v \in \mathbb{R}^{D \times D_v}$ and queries $\mathbf{q} \in \mathbb{R}^{M \times D_k}$:

$$\mathbf{k} = \mathbf{x}W_k, \quad \mathbf{v} = \mathbf{x}W_v, \quad e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{k}_i}{\sqrt{D_k}}, \quad \mathbf{a}_{\cdot,j} = \operatorname{softmax}(\mathbf{e}_{\cdot,j}), \quad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{v}_i \in \mathbb{R}^{D_v}.$$

Definition 10.7: self-attention layer

If the queries are also computed from the inputs, $\mathbf{q} = \mathbf{x}W_q$ with $W_q \in \mathbb{R}^{D \times D_k}$, the layer maps N inputs to N outputs and is called **self-attention**. In matrix form, with rows as tokens,

$$\mathrm{SA}(\mathbf{x}) = \mathrm{Attn}(\mathbf{x}W_q, \mathbf{x}W_k, \mathbf{x}W_v) = \operatorname{softmax}\Big(\frac{\mathbf{x}W_q(\mathbf{x}W_k)^\top}{\sqrt{D_k}}\Big)\mathbf{x}W_v,$$

where the softmax acts on each row. This is the **scaled dot-product attention** of the Transformer, $\mathrm{Attn}(Q, K, V) = \operatorname{softmax}(QK^\top/\sqrt{d_k})V$.

The matrix $A = \operatorname{softmax}(QK^\top/\sqrt{d_k}) \in \mathbb{R}^{n \times n}$ is row-stochastic, and row i of AV is a weighted average of all value vectors. So each output is a contextual encoding of the input whose receptive field is the whole sequence. This is the sense in which attention is **feature aggregation**.

The four steps in the lecture. With $\mathbf{x} \in \mathbb{R}^{n \times d}$ and projection matrices $W_q, W_k \in \mathbb{R}^{d \times d_k}$, $W_v \in \mathbb{R}^{d \times d_v}$, set $Q = \mathbf{x}W_q$ (the **query**, "the question"), $K = \mathbf{x}W_k$ (the **key**, "the index of the information") and $V = \mathbf{x}W_v$ (the **value**, "the information itself"). Then:

(i) $A = QK^\top \in \mathbb{R}^{n \times n}$, where $A_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$ measures how much token i takes the information of token j into account;

(ii) $A' = QK^\top/\sqrt{d_k}$ (scaling): if the entries of $\mathbf{q}, \mathbf{k}$ are independent with mean 0 and variance 1, then $z = \sum_{m=1}^{d_k} q_m k_m$ has $\mathbb{E}z = 0$ and $\operatorname{Var} z = \sum_m \operatorname{Var}(q_m k_m) = d_k$, so dividing by $\sqrt{d_k}$ restores unit variance;

(iii) $\alpha_{ij} = \operatorname{softmax}(A')_{ij}$, row by row;

(iv) $\mathrm{output}_i = \sum_j \alpha_{ij} \mathbf{v}_j$.

Cross versus self. In cross-attention the queries come from elsewhere (e.g. a decoder) and the keys and values from the input X; in self-attention Q, K and V all come from X. Attention-based captioning (Section 1) is cross-attention: the query is the state of the sentence being generated, and the keys and values are the image features. In the single-query layer of Definition 10.4 the keys and values coincide (both are $\mathbf{x}_i$).

5 Permutation Symmetry

Let P be a permutation matrix, so $P\mathbf{x}$ lists the same tokens in a different order. Cross-attention with fixed queries is **permutation invariant**: replacing $\mathbf{x}$ by $P\mathbf{x}$ reorders the keys and the values in the same way, so the weighted sums are unchanged. Self-attention is **permutation equivariant**: $\mathrm{SA}(P\mathbf{x}) = P\,\mathrm{SA}(\mathbf{x})$. Each score $\mathbf{q}_i \cdot \mathbf{k}_j$ depends only on which two tokens are compared, not on their positions, so reordering the inputs reorders the outputs in the same way. So self-attention alone treats a sentence as a set of words: "dog bites man" and "man bites dog" give the same outputs up to reordering. Word order has to be supplied separately.

6 Positional Encoding

Motivation. Self-attention is permutation equivariant, so it cannot see positions. We give each token a vector that encodes its position and attach it to the token's embedding.

Use a function $\mathrm{pos} : \mathbb{N} \to \mathbb{R}^d$ that turns position j into a d-dimensional vector $\mathbf{p}_j = \mathrm{pos}(j)$, and attach it to the input $\mathbf{x}_j$. The slides concatenate $\mathbf{p}_j$ to $\mathbf{x}_j$; the Transformer adds it, $\mathbf{x}_j + \mathbf{p}_j$. Two options for pos:

1. **Learn a lookup table**: a parameter vector $\mathrm{pos}(t)$ for each $t \in [0, T)$, so $T \times d$ parameters.
2. **Design a fixed function** with the desired properties.

The intuition for the fixed design is a binary counter. Writing $0, 1, 2, \ldots, 15$ in binary, the lowest bit alternates with period 2, the next with period 4, and so on. Each bit is a periodic function of the position, at a different frequency.

Definition 10.8: sinusoidal positional encoding

For even d and $k = 0, 1, \ldots, \frac{d}{2} - 1$, let $\omega_k = 1/10000^{2k/d}$ and

$$\mathrm{PE}(t)_{2k} = \sin(\omega_k t), \qquad \mathrm{PE}(t)_{2k+1} = \cos(\omega_k t).$$

Equivalently, $\mathrm{PE}(t) = \big(\sin\omega_0 t, \cos\omega_0 t,\ \ldots,\ \sin\omega_{d/2-1} t, \cos\omega_{d/2-1} t\big)$.

Any periodic functions can be used, or the encoding can be learned. Unlike a lookup table, the sinusoidal encoding is defined for every position t. The frequencies decrease geometrically, from $\omega_0 = 1$ down to about $1/10000$, like the bits of a binary counter: the first coordinates change quickly with the position and the last ones slowly.

Example 10.9: $d = 4$ (lecture example)

With $d = 4$ there are two frequencies, $\omega_0 = 1$ and $\omega_1 = 1/10000^{2/4} = 0.01$, so $\mathrm{PE}(t) = (\sin t, \cos t, \sin 0.01t, \cos 0.01t)$ and

$$\mathrm{PE}(0) = (0, 1, 0, 1), \qquad \mathrm{PE}(1) \approx (0.841, 0.540, 0.010, 1.000),$$
$$\mathrm{PE}(2) \approx (0.909, -0.416, 0.020, 1.000).$$

The first pair changes quickly with t and the second slowly, like the low and high bits of a binary counter. With the encoding added, $\mathbf{z}_j = \mathbf{x}_j + \mathbf{p}_j$, a token keeps its embedding but shifts by its position: for $\mathbf{x}_{\mathrm{dog}} = (1, 0, 2, 0)$,

$$\mathbf{x}_{\mathrm{dog}} + \mathrm{PE}(0) = (1, 1, 2, 1), \qquad \mathbf{x}_{\mathrm{dog}} + \mathrm{PE}(2) \approx (1.909, -0.416, 2.020, 1.000),$$

so “dog” at position 0 and at position 2 are different inputs, and “dog bites man” is no longer a reordering of “man bites dog”.

7 Masked Self-Attention

When generating text, the model must not see future words. The lecture’s example: training on “man plays guitar”, the Transformer reads the whole shifted sentence at once and predicts every next token in parallel.

position	input	target
0	<SOS>	man
1	man	plays
2	plays	guitar
3	guitar	<EOS>

At position 1 the target is “plays”; if position 1 could attend to position 2, whose input is “plays”, it would copy the answer. So a query at position j may only attend to positions $0, \dots, j$. In self-attention, set the alignment scores from future positions to $-\infty$ by hand. Since $e^{-\infty} = 0$, those positions get zero weight.

Definition 10.10: masked (causal) self-attention

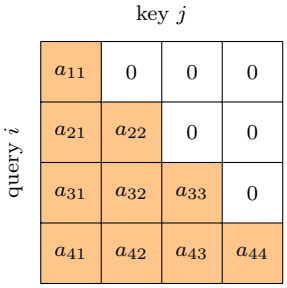
With $\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j$ as in Definition 10.7, query i uses the scores

$$e_{i,j} = \begin{cases} \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_k}, & j \le i, \\ -\infty, & j > i, \end{cases} \qquad \mathbf{y}_i = \sum_j a_{i,j} \mathbf{v}_j, \quad \mathbf{a}_{i,\cdot} = \operatorname{softmax}(\mathbf{e}_{i,\cdot}).$$

The slides index the same matrix the other way round, with the key index first: $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D_k}$ for $i \le j$ and $-\infty$ for $i > j$, so for three tokens

$$E = \begin{pmatrix} e_{0,0} & e_{0,1} & e_{0,2} \\ -\infty & e_{1,1} & e_{1,2} \\ -\infty & -\infty & e_{2,2} \end{pmatrix}$$

(column j holds the scores of query j), the transpose of the lower-triangular pattern below. Every row of the attention matrix still sums to 1, and the output of position i uses only the tokens $1, \dots, i$.



A causal attention matrix for four tokens: each query sees only itself and earlier tokens.

8 Multi-Head Attention

Definition 10.11: multi-head attention

With h heads, each with its own projections $W_q^{(r)}, W_k^{(r)} \in \mathbb{R}^{d\times d_k}$ and $W_v^{(r)} \in \mathbb{R}^{d\times d_v}$,

$$\text{head}_r = \text{Attn}\left(QW_q^{(r)}, KW_k^{(r)}, VW_v^{(r)}\right), \qquad \text{MHA}(Q,K,V) = [\text{head}_1, \ldots, \text{head}_h]\, W_o,$$

with $W_o \in \mathbb{R}^{hd_v\times d}$. Usually $d_k = d_v = d/h$.

Each head runs scaled dot-product attention (MatMul, Scale, optional Mask, SoftMax, MatMul) on its own linear projections, and the results are concatenated and mixed by a final linear layer. Different heads can attend to different relations, for example one to the previous word and one to the subject of the sentence.

9 The Transformer Encoder

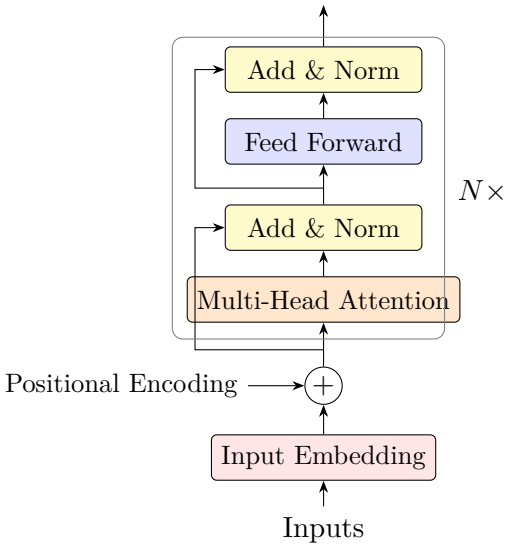
Motivation. The Transformer (Vaswani et al., 2017) stacks self-attention and position-wise MLPs, each with a skip connection and layer normalization.

Definition 10.12: Transformer encoder

1. The input is tokenized, $\mathbf{x} \in \mathbb{R}^{n\times d}$ after each token is encoded into a continuous embedding.
2. A positional encoding is added: $X_0 = \mathbf{x} + \text{PE}$.
3. Each of N layers applies multi-head self-attention with a skip connection, then a feed-forward network with a skip connection:

$$Z = \text{LN}\left(X + \text{MHA}(X,X,X)\right), \qquad X' = \text{LN}\left(Z + \text{FFN}(Z)\right),$$

   where $\text{FFN}(\mathbf{z}) = W_2\,\text{ReLU}(W_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$ is applied to each token separately.
4. The output of one layer is the input of the next.



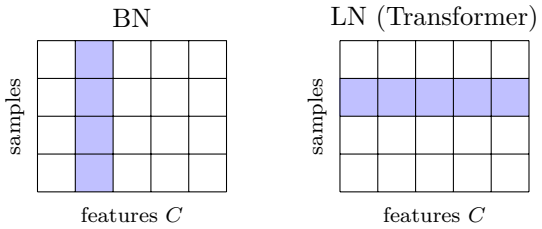
One encoder layer (inside the box), repeated N times; the side arrows are the skip connections.

Without PE every layer is still permutation equivariant, so positions enter only through PE. The cost of a layer is dominated by $QK^\top \in \mathbb{R}^{n\times n}$: $O(n^2 d)$ time and $O(n^2)$ memory in the sequence length n.

10 Layer Normalization

Batch normalization (Chapter 6) normalizes each channel using statistics over the mini-batch. Layer normalization normalizes each sample by itself.

- **BN**: for each channel C, average over the batch N and the positions H, W.
- **LN in CNNs**: for each sample, average over C, H, W.
- **LN in Transformers**: for each sample and each token, average over the C features only.



The shaded cells share one mean and variance: one feature over all samples (BN) versus all features of one sample (LN).

Definition 10.13: layer normalization

For $\mathbf{x} \in \mathbb{R}^d$, let $\mu = \frac{1}{d}\sum_k x_k$ and $\sigma^2 = \frac{1}{d}\sum_k (x_k - \mu)^2$. Then

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu \mathbf{1}}{\sqrt{\sigma^2 + \epsilon}}, \qquad \mathrm{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \hat{\mathbf{x}} + \boldsymbol{\beta},$$

with learnable $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ and a small $\epsilon > 0$.

11 Why Transformers

The original Transformer was developed for machine translation. It turned out to scale surprisingly well with large datasets, and most large-scale models since (BERT, GPT-2, Megatron-LM, T5, Turing-NLG, with up to 17 billion parameters by early 2020) are built on it. Its weak inductive bias (only PE carries order) is a cost on small data and an advantage once data is plentiful.